

1 Coordinate frames

- $\mathcal{F}_{cam}$: a 2D camera-plane frame aligned with the external camera image. All global mapping and planning are performed in $\mathcal{F}_{cam}$.
- $\mathcal{F}_{base}$: robot base frame (`base.link`).
- $\mathcal{F}_{laser}$: lidar frame rigidly attached to $\mathcal{F}_{base}$ via a fixed transform.

MVP. All planning and control are performed directly in the camera image plane

Baseline. A trained translation is introduced to ensure consistent motion execution and controller tuning.

Extensions. An online refinement of the scaling using motion consistency enables self-calibration without explicit camera setup, possibly introducing state-awareness / uncertainty.

2 State, action, observations, and map

2.1 State

The robot planning state is defined as its pose in the camera-plane frame:

$$s_k = \begin{bmatrix} x_k^{cam} \\ y_k^{cam} \\ \theta_k \end{bmatrix}. \quad (1)$$

MVP. The state s_k is obtained from simulator ground truth projected into $\mathcal{F}_{cam}$.

Baseline. Vision-based tracking of the robot in the external camera image. Speed and acceleration? Assume command = speed?

Extensions. perception model may be optimized or learned end-to-end, e.g., using segmentation, keypoint detection, or learned visual embeddings.

2.2 Action

The control input consists of commanded linear and angular velocities:

$$u_k = \begin{bmatrix} v_k \\ \omega_k \end{bmatrix}, \quad v_k \in [0, v_{\max}], \quad \omega_k \in [-\omega_{\max}, \omega_{\max}]. \quad (2)$$

MVP. Velocity commands are generated by a simple path-following controller.

Baseline. Convert geometric tracking errors into velocity commands.

Extensions. Learned policies or inferred by minimizing an expected free energy objective.

2.3 Observations

$$g_k^{cam} = \begin{bmatrix} x_{g,k}^{cam} \\ y_{g,k}^{cam} \end{bmatrix}, \quad (\text{goal location in } \mathcal{F}_{cam}), \quad (3)$$

$$\hat{s}_k = s_k + n_k^{pose}, \quad (\text{robot pose estimate in } \mathcal{F}_{cam}), \quad (4)$$

where n_k^{pose} denotes pose estimation noise induced by the camera-based perception pipeline.

MVP. In simulation, perfect robot pose estimates in $\mathcal{F}_{cam}$ and noiseless lidar measurements are used.

Baseline. Realistic noise is introduced in camera-based pose estimation, camera obstacle detection, and lidar range measurements.

Extensions. Camera-based pose and obstacle observations are augmented with uncertainty estimates whose magnitude depends on viewpoint, occlusion, and image quality.

2.4 Planning

At each time step, the planner computes a reference trajectory (e.g., a waypoint sequence) in the camera-plane frame:

$$\pi_k = \text{Plan}(c_k, \hat{s}_k, g_k^{cam}). \quad (5)$$

The planner consumes the current belief about traversability encoded in the costmap c_k , the current pose estimate $\hat{s}_k$, and the goal location g_k^{cam} .

MVP. No global planning is performed. The reference is a straight-line target in the image plane:

Baseline. Grid-based planning is performed on a costmap derived from the occupancy grid.

Extensions Replace $\text{Plan}(\cdot)$ with policy/trajectory inference by minimizing an Expected Free Energy-like objective over candidate trajectories (or control sequences).

2.5 Map

An occupancy grid map is maintained over a discrete grid $\mathcal{G}$ in the camera-plane frame $\mathcal{F}_{cam}$:

$$m_k : \mathcal{G} \rightarrow [0, 1], \quad (6)$$

where each cell stores a belief that the corresponding camera-plane region is occupied or non-traversable. A costmap c_k is derived from m_k via obstacle inflation to account for the robot footprint and safety margins.

MVP. No map is maintained

Baseline. A binary occupancy grid is constructed from obstacle observations (camera-based obstacle detection and/or lidar-based ray casting), and inflated with a fixed radius to form c_k .

Extensions. The map is extended from occupancy to a probabilistic traversability belief, e.g., (i) semantic labels (floor, obstacle, unknown) or (ii) learned traversal costs from vision. Map uncertainty (unknown/unobserved or high-entropy regions) is explicitly represented and incorporated into planning.

3 Motion model

The robot follows a discrete-time unicycle model in $\mathcal{F}_{cam}$:

$$s_{k+1} = f(s_k, u_k) + w_k, \quad w_k \sim \mathcal{N}(0, Q), \quad (7)$$

with

$$f(s_k, u_k) = \begin{bmatrix} x_k^{cam} + v_k \Delta t \cos \theta_k \\ y_k^{cam} + v_k \Delta t \sin \theta_k \\ \theta_k + \omega_k \Delta t \end{bmatrix}. \quad (8)$$

The motion model serves as the predictive component required by both classical planning (rollout and tracking) and planning-as-inference (predicting future states—and hence future observations—under candidate actions), analogous to the transition model used in Kouw.

MVP. Dynamics are handled by Gazebo entirely.

Baseline. Process noise, actuation limits, and (optionally) rate limits on (v_k, ω_k) are modeled explicitly.

Extensions. Learned or adaptive transition models replace fixed kinematics to reduce model mismatch (e.g., slip, delays).

4 Objective and success condition

$$\left\| \begin{bmatrix} x_k^{cam} \\ y_k^{cam} \end{bmatrix} - g_k^{cam} \right\|_2 \leq \epsilon_g. \quad (9)$$

MVP. Goal-reaching success rate.

Baseline. Goal reached based on prior

Extensions. Trajectory cost minimization under safety constraints.

5 Starting point

The project should start with the MVP consisting of:

- TurtleBot3 that listens to velocity commands
- Camera that sees the turtlebot
- ground-truth robot pose projected into $\mathcal{F}_{cam}$,
- Arbitrary plans on camera frame sent to robot
- Plans are translated accordingly
- pure pursuit path-following control,

CHATGPT: Closed-loop pipeline interpretation.

1. Observation acquisition.

$$o_k \{I_k, z_k^{lid}, g_k^{cam}\},$$

consisting of the external camera image I_k , the onboard lidar scan z_k^{lid} , and the goal location g_k^{cam} expressed in the camera-plane frame.

2. Vision-based pose observation.

The vision module maps the camera image to a pose observation and uncertainty,

$$(\hat{s}_k, \Sigma_k^{pose}) = \mathcal{V}_\theta(I_k),$$

which is interpreted as a likelihood model

$$p(\hat{s}_k \mid s_k) \approx \mathcal{N}(\hat{s}_k; s_k, \Sigma_k^{pose}).$$

In contrast to classical sensors, the covariance Σ_k^{pose} may depend on image conditions (viewpoint, occlusion, blur), inducing state-dependent observation uncertainty.

3. State belief prediction.

The belief over the robot state is first predicted forward using the motion model and the previously executed (safe) command:

$$p(s_k \mid o_{1:k-1}) = \int p(s_k \mid s_{k-1}, u_{k-1}^{safe}) p(s_{k-1} \mid o_{1:k-1}) ds_{k-1}.$$

4. State belief update.

The predicted belief is updated using the vision likelihood:

$$p(s_k \mid o_{1:k}) \propto p(\hat{s}_k \mid s_k) p(s_k \mid o_{1:k-1}).$$

For tractability, the posterior is approximated as a Gaussian

$$p(s_k \mid o_{1:k}) \approx \mathcal{N}(\mu_k, \Sigma_k).$$

5. **Map belief update.** In log-odds form,

$$\ell_k(i) = \ell_{k-1}(i) + \log \frac{p(\text{occ}_i \mid z_k^{lid}, \mu_k)}{1 - p(\text{occ}_i \mid z_k^{lid}, \mu_k)} - \ell_0,$$

followed by $m_k(i) = \sigma(\ell_k(i))$. Errors in the pose belief directly affect map consistency through misregistration.

6. **Costmap construction.** The occupancy belief is converted into a planning representation via inflation:

$$c_k = \text{Inflate}(m_k, r_{\text{robot}}, d_{\text{safe}}),$$

encoding both obstacles and safety margins.

7. **Planning.** *Baseline planning* computes a deterministic path:

$$\pi_k = \text{Plan}(c_k, \mu_k, g_k^{cam}).$$

Planning as inference instead evaluates candidate control sequences $\bar{u}_k = (u_k, \dots, u_{k+H-1})$ by predicting future state distributions $q(s_t \mid \bar{u}_k)$ and minimizing the Expected Free Energy

$$G(\bar{u}_k) = \sum_t \underbrace{E[-\log \hat{p}(s_t)]}_{\text{risk}} + \underbrace{E[H(p(o_t \mid s_t))]}_{\text{ambiguity}}.$$

Here, the risk term penalizes distance to the goal and high-cost map regions, while the ambiguity term penalizes trajectories expected to yield unreliable observations.

8. **Control extraction.** A velocity command is obtained either by tracking the planned path,

$$u_k = \text{Track}(\pi_k, \mu_k),$$

or by executing the first action of the inferred control sequence,

$$u_k = \text{first}(\bar{u}_k^*).$$

9. **Execution and state evolution.**

$$s_{k+1} = f(s_k, u_k) + w_k,$$

closing the perception–action loop.